

Patrones de Diseño en Go

Por: Ricardo Cuellar



¿Qué son los patrones de diseño?

Son soluciones reutilizables a problemas comunes en diseño de software.

Son ideas, estructuras o formas de organizar el código para resolver problemas que aparecen muchas veces en distintos proyectos.

Factory

Adapter

Repository

Strategy



¿Para qué sirven?

- Organizar mejor el código
- Separar responsabilidades
- Reducir el acoplamiento
- Facilitar pruebas
- Hacer el código más flexible ante cambios
- Mejorar la comunicación entre desarrolladores



Go es diferente

Los patrones se hicieron populares con lenguajes orientados a objetos como Java, C# y C++.

- Go prefiere composición sobre herencia
- No tiene clases
- No tiene herencia tradicional
- Usa interfaces pequeñas
- Favorece código simple y explícito
- No se busca la sobreingeniería



Go es diferente

Go si puede usar patrones, pero se usan de forma más simple y es menos estricto que seguir la definición de libro.

En vez de: *¿Qué patrón podemos meter aquí?*

Se debe pensar: *¿Qué problema real tenemos y cuál es la forma más simple de resolverlo?*



¿Cuándo debe usarse?

- Resuelve un problema real
- Hace el código más claro
- Reduce la duplicación
- Mejora las pruebas
- Ayuda a cambiar el comportamiento sin romper otras partes
- Evita que el código crezca desordenado



¿Cuándo NO debe usarse?

- Solo para vernos profesionales
- Complicar casos simples
- Crear muchas capas de forma innecesaria
- Si hace el código más difícil de entender
- Se mete antes que exista el problema



¿Cómo se usan los patrones en Go?

```
example.go

type TaskRepository interface {
    FindByID(id string) (*Task, error)
    Save(task *Task) error
}
```

Interfaces pequeñas

```
example.go

type TaskService struct {
    repo TaskRepository
}
```

Estructuras



¿Cómo se usan los patrones en Go?



example.go

```
type ValidatorFunc func(input string) error
```

Funciones



example.go

```
type Server struct {  
    logger *slog.Logger  
    db     *sql.DB  
    router http.Handler  
}
```

Composición



Patrones útiles en Go

- Patrón repository
- Patrón strategy
- Patrón adapter
- Patrón middleware



Inyección de dependencias

Algunos no lo consideran un patrón pero es importante en Go. Sirve para pasar dependencias desde afuera en lugar de crearlas dentro del componente.



No todos los patrones son útiles

Algunos patrones no podrán ser útiles en nuestros programas en Go debemos ser cuidadosos el como queremos aplicarlos sin caer en complejidad excesiva.



Recomendación para usar patrones en Go

Primero haz el código simple y después el patrón.

Si con el código simple se soluciona estás del otro lado, si hay dolor o problemas entonces hay que refactorizar.

Señales:

- Mucho código duplicado
- Mucho switch o if
- Handler enormes
- Lógica de negocio mezclada
- Acomplamiento agresivo



Gracias



www.devtalles.com



[/ricardocuellars](https://www.linkedin.com/company/ricardocuellars)



[@ricardocuellars](https://twitter.com/ricardocuellars)

